

Cron at the center of your pipelines

Still trusting your business to a crontab file? Time to wake up.

The context

We all love Cron. It's simple, it's native, and it's been running for decades. But in modern engineering, Cron's "simplicity" is a security and reliability risk. If your Cron task fails at 3 AM:

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="job", retry_count=3, retry_delay=10)
def job(data):
    return {"done": run(data["command"])}

pipe = Pipeline(pipeline_name="cron_pro", tracking_db="cron.db")
pipe.set_steps([job])

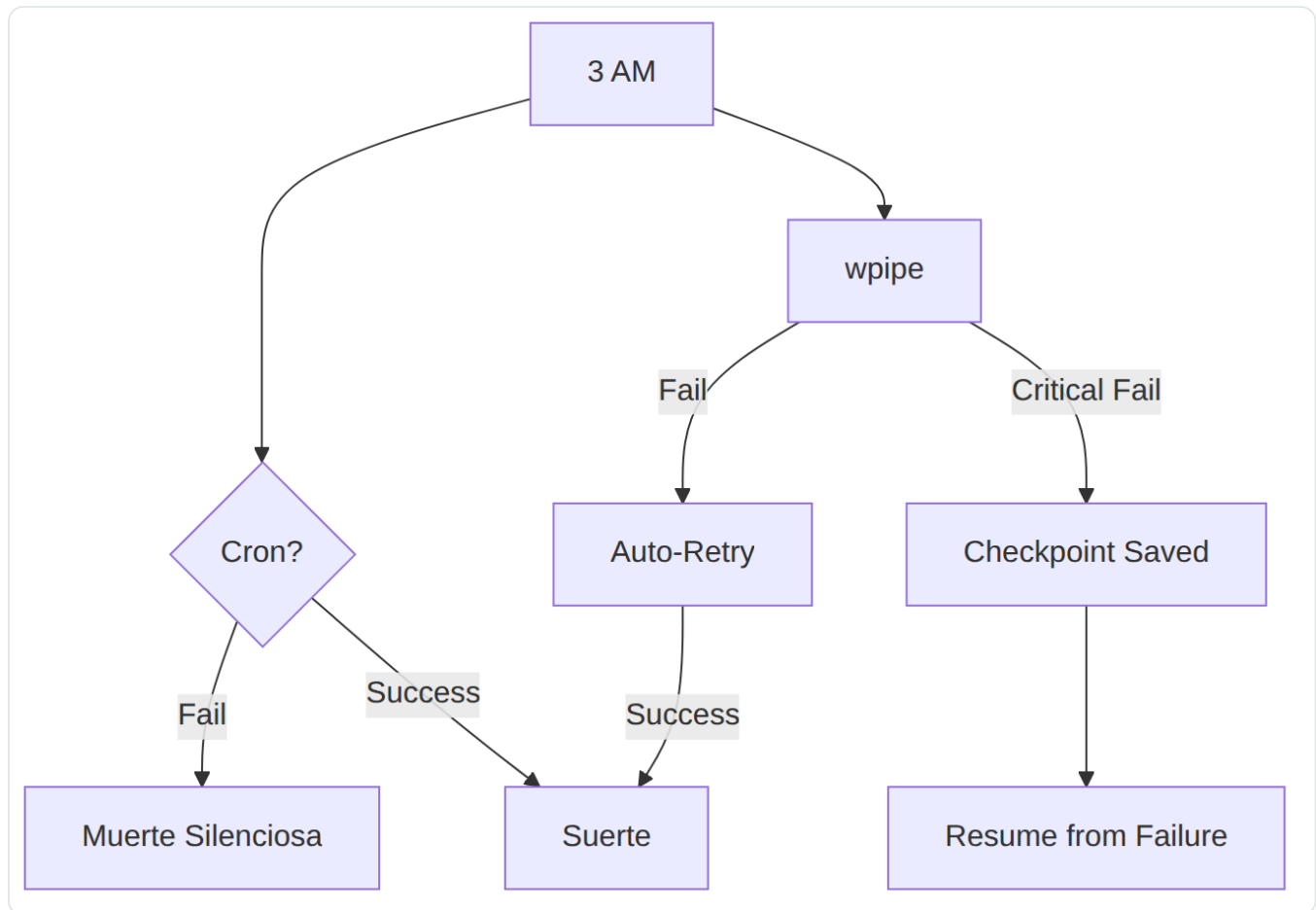
# Keep the schedule; gain checkpoints + audit.
```

At a glance

Challenge	Cron (Legacy)	wpipe (Modern)
Visibility	Silent Failures	Forensic (SQL Tracker)
Resilience	Start from zero	Checkpoints (Save Game)
Complexity	Fragile scripts	Modular code (@step)
Alerts	Opaque system logs	Native alert integration

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Cron** workflows at wpipe.readthedocs.io.